
Food Recognition with Custom CNNs: Supervised Training versus Self-Supervised Pretraining on iFood-2019

Angelica Iseni

Kevin Del Gaudio

Abstract

We study fine-grained food recognition on the iFood-2019 dataset (251 classes, 118,475 training images with a highly variable resolution) under a 10M parameter budget. Two custom convolutional networks trained from scratch, a plain five-block CNN (1.04M parameters) and a residual network (5.03M, trained with two dropout rates, 0.3 and 0.5), are compared with a third model built by transfer learning, an ImageNet-pretrained EfficientNet-B0 (4.33M) with the last blocks fine-tuned. The same residual encoder is then pretrained without labels with two pretext tasks, image colorization and SimCLR-style contrastive learning, frozen, and evaluated through linear and nearest-neighbour probes on the extracted features. All models are trained on a capped and stratified split of the training set and evaluated on the official validation set, which we never use for model selection. On this held-out set the plain CNN reaches 53.1% accuracy, the residual network 57.8% with its default dropout of 0.3 and 58.7% when dropout is raised to 0.5, and the fine-tuned EfficientNet-B0 64.9%. Self-supervised features are far weaker under linear probing, 27.6% for colorization and 31.6% for contrastive learning, but both are more than sixty times above chance and the contrastive encoder is consistently ahead.

1 Introduction

Recognising a dish from a photo is a fine-grained classification problem: many classes differ only in small details (cannelloni versus lasagna, coffee cake versus marble cake), the same dish is photographed under very different lighting, angles and plating, and colour carries a lot of the useful information. The iFood-2019 challenge [1], built on the FoodX-251 dataset [2], makes the problem harder still by having 251 classes, an uneven number of images per class and web-collected training images whose labels are noisy, while the validation and test labels were verified by humans.

First, an image classifier is built as a custom CNN with fewer than 10M parameters, trained in a supervised way and evaluated with accuracy, precision, recall and F1. Second, the same kind of encoder is trained in a self-supervised way on the same images with the labels ignored, followed by feature extraction and a traditional classifier on top, so that the quality of the two kinds of representation can be compared.

Section 2 covers the data, Section 3 the models and the training protocol, Section 4 the results, Section 5 the limitations and future work.

2 Data and preprocessing

Dataset and splits. The competition provides a training set of 118,475 images, a validation set of 11,994 images and a test set without public labels. The official validation set is used as our test set and is touched only once per model for the final numbers; a validation split for early stopping and checkpoint selection is carved out of the training set instead. The class list contains 251 dishes and every class appears in both sets.

Class distribution. The training set is less unbalanced than we expected: the number of images per class ranges from 34 to 656 with mean 472 and median 471, and most classes sit between 300 and 600 images. The richest classes (huitre, casserole, churro, macaron) have around 600 images, the poorest (lobster thermidor, matzo ball, scrapple, welsh rarebit) still above 300, with the exception of marble cake, which is a clear outlier at the bottom. To reduce the gap and, at the same time, to keep the training time within our Colab budget, we cap every class at the 60th percentile of the class counts, 500 images. The cap only trims the 100 classes above it, leaving the small classes untouched, and brings the number of training images down to 113,325. All 251 classes are kept, as required.

Train and validation split. The capped set is split with stratification on the label, 85% for training and 15% for validation, with a fixed seed, giving 96,326 training and 16,999 validation images. Stratification matters because of the small classes: with a plain random split a class with 34 images could easily end up with two or three validation images.

Image properties. Since opening all images only to collect statistics would take too long, sizes, aspect ratios, colour modes and file integrity were checked on random samples of 200 to 500 images. Every image is at least 256×256 pixels, most sit close to that minimum, with a mean width of 347 and a mean height of 285 pixels and a long tail up to 772×1024 . The aspect ratio is mostly between 1.2 and 1.6, so images are slightly landscape and a centre crop does not cut away much of the plate. All sampled images are RGB and none is corrupted; a conversion to RGB and a try-except on image opening are nevertheless kept in the data loader as a safety net.

Resizing and normalisation. Every image is resized so that its shorter side is $1.15 \times 224 = 257$ pixels and then cropped to 224×224 , a size that sits where most of the mass of the size distribution is and that keeps the cost of a 224×224 forward pass manageable on a single GPU. Pixel values are scaled to $[0, 1]$ and standardised with the ImageNet mean and standard deviation per channel. For the two networks trained from scratch this is simply a reasonable zero-mean, unit-variance normalisation; for the fine-tuned backbone it is the normalisation the pretrained weights were computed with and is therefore required.

Augmentation. Training images take a random 224×224 crop and a horizontal flip with probability 0.5; validation and test images take a centre crop and nothing else. No colour augmentation is used in the supervised pipeline on purpose, because colour is one of the strongest cues for food (a marble cake is a coffee cake with a different colour pattern). The contrastive pretext task is the one deliberate exception and is discussed in Section 3.5.

3 Methodology

3.1 Shared training protocol

All supervised models are trained with the same wrapper, written in PyTorch and Lightning [5]; the probes and the metrics use scikit-learn. The loss is cross-entropy with label smoothing 0.1, chosen because the training labels of this dataset are noisy and smoothing stops the network from becoming overconfident on wrong labels. Weight decay is applied only to weight matrices and convolution kernels, not to biases and batch-norm parameters, which is the usual practice and avoids shrinking parameters that have no regularising role. Batches hold 64 images. Training runs for at most 150 epochs and is stopped early when the validation accuracy does not improve by at least 0.5 points for 10 consecutive epochs (15 for the fine-tuned model); the checkpoint with the best validation accuracy is the one evaluated on the test set. Every random seed is fixed to 42.

Two optimiser settings are used. The plain CNN uses Adam with learning rate 10^{-3} and ReduceLROnPlateau on the validation accuracy (factor 0.5, patience 3). The two deeper models use AdamW with a linear warm-up followed by cosine annealing to 10^{-6} : the warm-up avoids large, unstable updates in the first epochs of a residual network, and the cosine schedule gives a smooth decay over the epoch budget instead of one tied to a validation plateau. Weight decay is 10^{-4} and dropout on the pooled feature vector is 0.3 by default; Section 3.3 also reports a variant of the residual network trained with dropout 0.5, everything else unchanged, as a regularisation ablation.

3.2 Base net: a plain CNN

The first model is the simplest architecture that could work: five blocks of a 3×3 convolution, batch normalisation, ReLU and 2×2 max pooling, with channels 32, 64, 128, 256, 256, followed by global average pooling, dropout and a linear layer to 251 classes. The 224×224 input is halved five times down to 7×7 . Global average pooling instead of flattening keeps the classifier small (256 inputs). The total is 1.04M trainable parameters, about a tenth of the budget, which makes this a deliberate lower bound.

3.3 Medium net: a residual CNN

The second model keeps the same overall shape, a stem followed by four stages, global average pooling, dropout and a linear classifier, but each stage is a residual block of two 3×3 convolutions with batch normalisation, summed with a 1×1 projected skip connection when the channel count changes, and followed by 2×2 max pooling. Channels are doubled at every stage, 64, 128, 256, 512, so the network is both deeper and wider than the base net and the skip connections let the gradient flow through the extra depth. The parameter count is 5.03M, half of the budget, leaving room for the decoder and the projection head that the self-supervised experiments add on top of the same encoder (Section 3.5). The encoder part of this model, everything up to the last stage, is the encoder reused in the self-supervised experiments. To check how much of the overfitting described in Section 4 is a regularisation issue, the same architecture, data and optimiser settings are also trained with dropout raised from 0.3 to 0.5 on the pooled feature vector; everything else, including the random seed, is unchanged. This second run does not change the parameter count and is referred to as *Medium net (dropout 0.5)* below.

3.4 Complex net: transfer learning with EfficientNet-B0

The third model uses transfer learning. The backbone is an EfficientNet-B0 pretrained on ImageNet, taken from torchvision, with its classifier replaced by dropout and a new linear layer to 251 classes. We chose this backbone because it is the largest widely used ImageNet model that fits the 10M budget with room to spare: the whole network has 4.33M parameters, less than the medium net. The first five feature blocks, which capture generic edges, textures and colour patterns, are frozen (0.31M parameters); the last four blocks and the new head, which need to adapt to food specific features, stay trainable (4.02M). Because the backbone starts from a good initialisation, the learning rate is ten times lower (10^{-4}) and the warm-up shorter (3 epochs) than for the medium net, so that the pretrained weights are adapted rather than destroyed. Only the parameters with gradients are handed to the optimiser.

3.5 Self-supervised pretraining

The self-supervised experiments use the medium net encoder (stem plus four residual stages, 4.90M parameters) trained on the training split with the label column never read. The encoder returns the $512 \times 7 \times 7$ feature map after the last stage; global average pooling is applied only afterwards, when the frozen encoder is used as a feature extractor. In both cases the extra modules needed by the pretext task are discarded after pretraining, so the 10M budget is checked on the encoder alone.

Colorization. The images are converted to the Lab colour space: the lightness channel L is the input and the two chroma channels (a, b) are the regression target. Lab is preferred to RGB because no single RGB channel is a grayscale image, whereas L is exactly one. L is rescaled to $[0, 1]$ and (a, b) to about $[-1, 1]$. The encoder is identical to the medium net one except for a single-channel stem. A decoder of five nearest-neighbour upsampling and 3×3 convolution blocks (1.57M parameters, no

skip connections) brings the 7×7 map back to 224×224 with a final tanh, whose range matches the rescaled target. The loss is the mean squared error between predicted and true (a, b) . Only spatial augmentation (random crop and flip) is applied: any colour jitter would change the target itself.

Contrastive learning. The second pretext task is a SimCLR-style [4] setup. Each image is augmented twice, independently, with a random resized crop (scale 0.3 to 1.0 rather than the usual 0.08, since a very small crop of a plate seen from above may contain no food at all), a horizontal flip, colour jitter and random grayscale with probability 0.2. Here strong colour augmentation is needed, otherwise the two views are trivially matched by their colour histogram and the encoder learns nothing. The pooled 512-dimensional encoder output goes through a two-layer projection head (512 to 512 to 128, 0.33M parameters) and the NT-Xent loss is computed on the projected vectors: for a batch of N images the $2N$ projections are ℓ_2 -normalised, their cosine similarity matrix is divided by a temperature $\tau = 0.5$, the diagonal is masked and each row is a cross-entropy over the $2N - 1$ remaining entries where the only positive is the other view of the same image.

With a batch of 64 every view has 126 negatives, far fewer than in the original paper, a limitation imposed by GPU memory and discussed in Section 5. Features for the downstream classifier are taken before the projection head, since the head absorbs part of the information the contrastive loss asks the network to discard.

Both pretext tasks use AdamW with learning rate 10^{-3} , cosine annealing over a budget of 100 epochs, early stopping on the validation loss (patience 10) and the checkpoint with the lowest validation loss.

Feature extraction and probing. After pretraining, the encoder is frozen and run in evaluation mode over the training split and the test set with the deterministic centre-crop transform, and the pooled 512-dimensional vector is stored for each image. Two classifiers are then trained on the training features and evaluated on the test features: a multinomial logistic regression (linear probe), which measures how linearly separable the classes are in the learned space, and a 5-nearest-neighbour classifier, which measures whether images of the same class already sit close together before any decision boundary is learned.

3.6 Evaluation

Every model is evaluated once on the 11,994 test images with accuracy, macro-averaged precision, recall and F1, and weighted F1. Macro averaging gives every class the same weight regardless of its size, so it is sensitive to the small classes that the cap left untouched; weighted F1 is reported alongside as the size-aware alternative. Errors are inspected with a confusion matrix restricted to the twenty classes with the most errors and with the per-class recall of the worst classes. With 251 classes the chance level is 0.4%.

4 Results

4.1 Supervised models

Table 1 summarises the four supervised runs, which form an increasing scale of depth, width, regularisation and initialisation quality, and Table 2 the test metrics of every model. The base net trains for 46 epochs and stops with training and validation accuracy still overlapping at 45% (Fig. 1, top): it underfits, it has simply run out of capacity. The medium net with its default dropout of 0.3 reaches its best validation accuracy, 50.0%, at epoch 28 and is stopped at 34; from epoch 15 onward the training accuracy keeps rising to 67% while the validation accuracy is flat and the validation loss starts to grow (Fig. 1, second panel), a clear overfitting pattern that dropout, label smoothing and weight decay slow down but do not remove. Raising dropout to 0.5 on the same architecture delays this pattern rather than removing it: the training accuracy still climbs, but only to 63%, the best validation accuracy rises slightly to 50.8% and is reached four epochs later, at epoch 32, and training runs for four more epochs before early stopping triggers, at epoch 38 (Fig. 1, third panel); the validation loss still drifts upward after its minimum, but more slowly than with dropout 0.3. The fine-tuned EfficientNet-B0 converges in a few epochs, 55% validation accuracy after five, peaks at 57.9% at epoch 18 and overfits sooner and harder (training accuracy above 80% by the end, Fig. 1, bottom), which is expected when only a few million parameters are adapted with a small learning rate on top of features that are already good.

Table 1: Supervised models: parameter budget, optimisation settings and training length. All runs use batch size 64, weight decay 10^{-4} and label smoothing 0.1. Trainable parameters are the ones handed to the optimiser.

Model	Architecture	Params	Trainable	Dropout	Optimiser	Schedule	Best ep.	Ep.
Base	5 conv blocks, 32 to 256 ch.	1.04M	1.04M	0.3	Adam, 10^{-3}	plateau	43	46
Medium (drop. 0.3)	stem + 4 residual stages	5.03M	5.03M	0.3	AdamW, 10^{-3}	warm-up 5, cosine	28	34
Medium (drop. 0.5)	stem + 4 residual stages	5.03M	5.03M	0.5	AdamW, 10^{-3}	warm-up 5, cosine	32	38
Complex	EfficientNet-B0, 5 frozen blocks	4.33M	4.02M	0.3	AdamW, 10^{-4}	warm-up 3, cosine	18	28

Table 2: Test-set metrics (11,994 images, 251 classes) for every model. Precision, recall and F1 are macro averages; weighted F1 weighs each class by its support. Self-supervised rows report the classifier trained on the frozen encoder features. Chance level is 0.4%.

Training	Model	Acc.	Prec.	Rec.	F1	wF1
Supervised	Base net	0.531	0.522	0.513	0.506	0.526
Supervised	Medium net (dropout 0.3)	0.578	0.587	0.564	0.561	0.577
Supervised	Medium net (dropout 0.5)	0.587	0.593	0.578	0.572	0.586
Supervised	Complex net (EfficientNet-B0)	0.649	0.640	0.638	0.632	0.647
Self-supervised	Colorization, linear probe	0.276	0.263	0.265	0.258	0.271
Self-supervised	Colorization, 5-NN	0.096	0.144	0.091	0.091	0.095
Self-supervised	Contrastive, linear probe	0.316	0.298	0.303	0.295	0.309
Self-supervised	Contrastive, 5-NN	0.122	0.154	0.114	0.110	0.117

On the test set the ranking is the same and the gaps are large: 53.1% for the base net, 57.8% for the medium net with dropout 0.3, 58.7% with dropout 0.5, and 64.9% for the fine-tuned backbone. Precision, recall and F1 move together with accuracy and macro F1 stays one to three points below accuracy in every case, which means that the small classes are recognised somewhat worse than the large ones but not dramatically so; weighted F1 is within half a point of accuracy everywhere. Going from the plain CNN to the residual one buys 4.7 points of accuracy for five times the parameters; raising the residual net’s dropout from 0.3 to 0.5 buys another 0.9 points without changing the parameter count, all of it on the regularisation side; ImageNet pretraining still buys the largest single jump, 6.2 points over the better-regularised residual net, with fewer trainable parameters than the medium net, a reminder of how much a good initialisation is worth on 96k noisy training images.

Validation versus test. For all four models the test accuracy is 7 to 8 points higher than the best validation accuracy (44.8% versus 53.1%, 50.0% versus 57.8%, 50.8% versus 58.7%, 57.9% versus 64.9%). Since the validation split comes from the same pool as the training images and the test set does not, the opposite would be the usual expectation. The most plausible explanation is label quality: the dataset authors [2] state that the training images were crawled from the web with noisy labels, including images of raw ingredients or of visually similar dishes, while the 40k validation and test images received human-verified labels. A model that has learned the visual content of a class is then scored against cleaner labels on the test set than on our validation split, and part of the apparent validation error is in fact label error. The gap is almost constant across models, so it does not change any of the comparisons, but it does mean that the validation numbers underestimate the real quality of the classifiers.

4.2 Error analysis

The confusion matrix of the best single model, restricted to the twenty classes with the most errors (Fig. 2), shows that the residual mistakes are between dishes that are visually and semantically close: marble cake is predicted as coffee cake in 7 of 17 cases, chicken provençale as rissole (9 of 14), tenderloin as pork chop, and lasagna and cannelloni are confused with each other. These are pairs that a human would also hesitate on from a single photo, and they are also pairs where the noisy training labels are likely to overlap. The worst classes for the base net, with a recall below 10%, are chicken provençale, marble cake, veal cordon bleu, confit and mostaccioli, all of them either rare (marble cake) or dishes without a fixed visual identity. Marble cake, the outlier of the class distribution, is worth a note: the base net predicts it correctly 2 times out of 49 but with perfect precision, so the

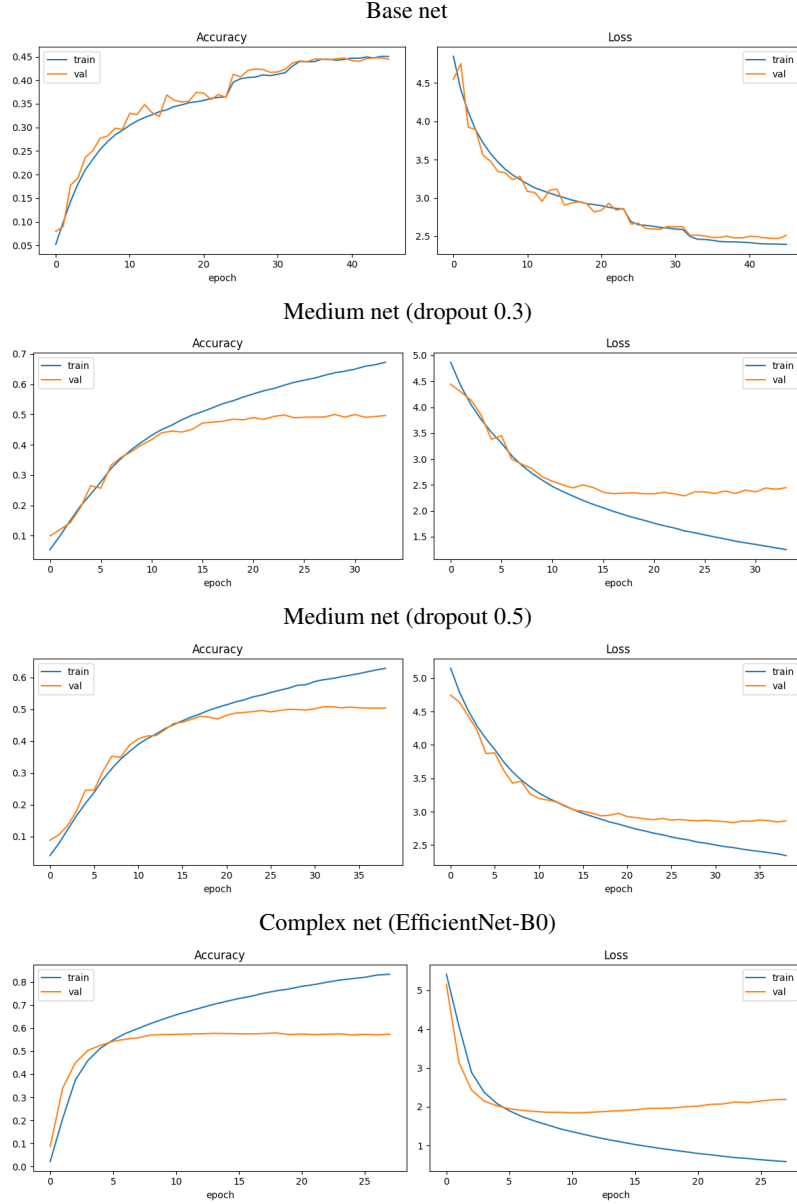


Figure 1: Accuracy (left panel) and loss (right panel) per epoch, on the training and validation splits, for the four supervised models. The base net (top) ends with the two curves still together, a sign of underfitting. The two medium net variants and the fine-tuned EfficientNet-B0 separate after roughly 15 and 5 epochs respectively, and their validation loss starts to rise while the training loss keeps falling, so the useful gains come from the early epochs and early stopping does the rest; raising dropout from 0.3 to 0.5 on the medium net narrows the train/validation gap without closing it.

network has learned a very narrow notion of the class, exactly what one expects from about thirty training images out of 96k.

4.3 Self-supervised representations

The colorization network reaches its lowest validation MSE, 0.0105 on the rescaled chroma channels, at epoch 21. Qualitatively (Fig. 3) the predicted colours are plausible but desaturated, with a tendency toward the average brown and beige of the dataset, which is the well-known behaviour of a mean-squared-error colorizer: when a dish could be red or green, the loss is minimised by a muddy average.

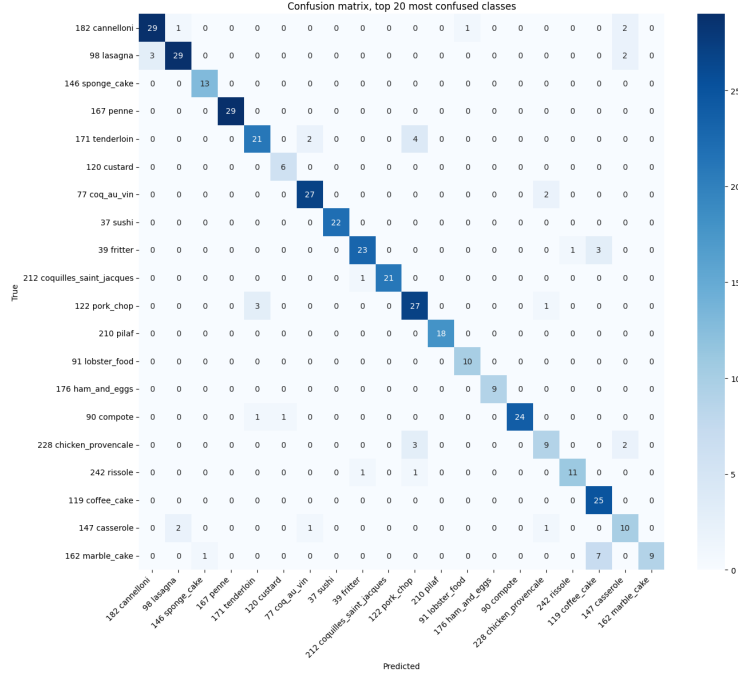


Figure 2: Confusion matrix of the fine-tuned EfficientNet-B0 on the test set, restricted to the twenty classes with the largest number of errors. Off-diagonal mass concentrates on semantically close pairs (marble cake and coffee cake, chicken provençale and rissole, tenderloin and pork chop, lasagna and cannelloni); most other errors fall outside this subset and are spread thinly over the remaining classes.

The contrastive network reaches its lowest validation NT-Xent loss, 3.12, at epoch 18. For reference, a loss of $\log(2N - 1) = \log 127 \approx 4.84$ corresponds to an encoder that cannot tell the positive from the negatives, so the network does learn to match the two views, but the loss is still far from zero.

Under linear probing the contrastive features reach 31.6% accuracy on the test set against 27.6% for the colorization features, and the same order holds on every metric and on the nearest-neighbour probe (12.2% versus 9.6%). Both are far above chance, which shows that a label-free pretext task on food images does produce class-relevant features, but both are also far below the 57.8% of the same encoder trained with labels. The nearest-neighbour probe is much weaker than the linear one for both encoders, so the classes are partly linearly separable but not clustered in the feature space; a plain Euclidean neighbourhood in 512 dimensions is a poor notion of similarity for these features.

Two things explain the difference between the two pretext tasks. First, colorization must, by construction, discard colour from its input, and colour is exactly one of the strongest cues in food recognition; the encoder learns shape and texture only, and its features are then evaluated on a grayscale version of the test image. Second, the contrastive task asks the encoder to be invariant to crop, flip and lighting changes while still telling different images apart, which is closer to what classification needs, and it keeps the full RGB input.

4.4 Hyperparameter tuning

Compute was the binding constraint of the project: every run of a deep model or of a pretext task costs several hours of a paid Colab GPU, so a grid search over all hyperparameters was out of reach. Tuning was therefore done as a sequence of experiments, each one changing a small set of hyperparameters and keeping the data pipeline fixed, with the validation accuracy as the selection criterion and the test set used only for the final number.

Architectural hyperparameters. The main axis of tuning is the architecture itself. Going from the base net to the medium net changes the depth (five plain blocks to a stem plus four two-convolution residual stages), the width (32 to 256 channels versus 64 to 512), the presence of skip connections and

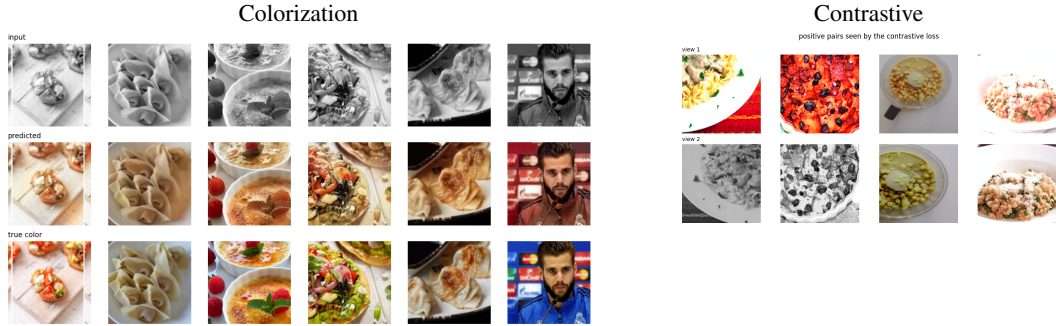


Figure 3: Left: grayscale input (top row), predicted colours (middle) and true colours (bottom) for six validation images of the colorization network; predictions are plausible but pulled toward desaturated averages. Right: four positive pairs seen by the contrastive loss, each column being two views of the same image; they differ in crop, flip, brightness and saturation while remaining recognisable as the same dish.

the parameter count (1.04M to 5.03M). The medium net’s width was set so that the encoder would fill about half of the budget and leave room for the decoder and projection head of the self-supervised tasks. The third step, the pretrained EfficientNet-B0, changes the initialisation and the block design while keeping the parameter count below the medium net. Each step improved the validation accuracy by 5 to 8 points (44.8%, 50.0%, 57.9%).

Optimisation hyperparameters. The base net used Adam at 10^{-3} with a plateau schedule; since it was still improving after 30 epochs, the epoch budget was raised to 150 for the deeper models and the schedule replaced by warm-up plus cosine, which gives a decay that does not depend on a noisy validation signal. For the fine-tuned model the learning rate was reduced tenfold and the early-stopping patience raised from 10 to 15 epochs, because a pretrained backbone converges in a few epochs and a longer patience protects against stopping on a flat but not yet peaked curve. Label smoothing 0.1 and weight decay 10^{-4} were fixed from the start as the standard regularisation set; dropout was fixed at 0.3 for the base and complex nets, and the medium net was additionally trained with dropout 0.5. For the pretext tasks the contrastive temperature was left at 0.5, the value used in the original paper with small batches, and the crop scale was moved from 0.08 to 0.3 after visual inspection of the positive pairs.

5 Discussion and conclusion

Limitations. The self-supervised comparison is not quite at parity with the supervised one. The contrastive encoder was pretrained with 126 negatives per view, an order of magnitude fewer than in the original work, on a 224×224 input, and the linear probe was trained once with default regularisation; larger batches, longer pretraining and a tuned probe would very likely narrow the gap, though not close it. The colorization encoder is evaluated on grayscale inputs, which handicaps it on a colour-driven task by design. On the supervised side, only one configuration per architecture was trained and no run was repeated with a different seed, so differences of one point or less should not be read as significant. The cap of 500 images per class removed 4% of the training data; we judged the time saved worth more than those images, but the larger classes may have lost some accuracy. Finally, the noisy training labels affect both the learning and our validation numbers, and the human-verified test set is the only reliable measure we have.

Future work. The most promising direction for the supervised models is fine-tuning the self-supervised encoder end to end on the labels instead of freezing it, which is how contrastive pretraining is normally used in practice and would tell whether the label-free initialisation helps a from-scratch network at all. Increasing the effective number of negatives, for instance with a memory bank, would test whether the small batch is the main limit of the contrastive run. Mild oversampling of the classes below 100 images, and a loss robust to label noise, would target the two data issues found in the error analysis.

Declaration of originality

We declare that this project, the code and this report, is our own work and that no part of it has been plagiarised. Every external idea, method, dataset and software library we relied on is cited in the references below.

References

- [1] iFood 2019 Challenge at CVPR FGVC6. Kaggle competition and dataset. <https://www.kaggle.com/c/ifood-2019-fgvc6>
- [2] P. Kaur, K. Sikka, W. Wang, S. Belongie, A. Divakaran. FoodX-251: a dataset for fine-grained food classification. *arXiv:1907.06167*, 2019.
- [3] R. Zhang, P. Isola, A. A. Efros. Colorful image colorization. In *ECCV*, pp. 649–666, 2016.
- [4] T. Chen, S. Kornblith, M. Norouzi, G. Hinton. A simple framework for contrastive learning of visual representations. In *ICML*, pp. 1597–1607, 2020.
- [5] W. Falcon and the PyTorch Lightning team. PyTorch Lightning. <https://lightning.ai>, 2019.